

Error Handling in C

Building Robust and Resilient Applications

Educational Resource by **CodeHive**

Volume 3: Systems Programming Series

1. Concept of Error Handling

In most modern high-level languages like Java or Python, error handling is managed through "Exceptions" using try-catch blocks. However, **C is a minimalist language**. It does not have built-in exception handling constructs.

In C, error handling is a manual process. The programmer must explicitly check the status of every function call and decide how the program should respond to failures. This requires a disciplined approach to coding.

Pro Tip: Professional C code often spends 40% of its lines handling "what ifs" and potential errors rather than just the core logic.

2. Categorizing Errors

2.1 Compile-Time Errors

These are the easiest to fix because the compiler (like GCC) will refuse to build the program until they are resolved. They include syntax mistakes and type mismatches.

```
int x = "Hello"; // Error: Incompatible types printf("%d", y); //  
Error: 'y' undeclared
```

2.2 Runtime Errors

These occur while the user is running your program. Logic that seemed fine during coding fails due to external factors like missing files or division by zero.

3. The 'errno' Global Variable

When a system call or a library function fails, it usually sets a global variable named `errno`. This is an integer representing a specific error code.

How to use `errno`:

To use it, you must include `<errno.h>`. It is good practice to reset `errno` to 0 before a function call that you want to check.

```
#include <errno.h> #include <string.h> // Example after a failed
operation printf("Error code: %d\n", errno); printf("Message:
%s\n", strerror(errno));
```

4. Reporting Errors: perror() vs strerror()

C provides two primary ways to turn numeric error codes into human-readable messages.

4.1 perror()

The `perror()` function prints a string you provide, followed by a colon, a space, and the actual error message corresponding to the current `errno` value.

```
FILE *fp = fopen("nonexistent.txt", "r"); if (fp == NULL)
{ perror("Error opening file"); } // Output: Error opening file:
No such file or directory
```

4.2 strerror()

The `strerror()` function (found in `<string.h>`) returns a pointer to the textual representation of the current `errno`. This is more flexible as it allows you to format your own string.

```
fprintf(stderr, "Log Error: %s", strerror(errno));
```

5. Handling Exit Status

When a program finishes, it returns a status code to the Operating System. By convention:

- **EXIT_SUCCESS (0):** The program finished perfectly.
- **EXIT_FAILURE (1 or non-zero):** The program encountered an error.

```
#include <stdlib.h> if (critical_error) { exit(EXIT_FAILURE); }
```

6. File Operations Error Handling

The `fopen` function returns `NULL` if it fails. Always check this before proceeding to read/write.

```
FILE *file = fopen("config.sys", "rb"); if (!file)
{ perror("Config load failed"); return -1; }
```


7. Memory Allocation

Safeguards

Memory is a finite resource. In embedded systems or high-load servers, malloc will eventually fail.

Never assume memory is allocated. Accessing a NULL pointer returned by malloc results in a **Segmentation Fault**.

```
int *buffer = malloc(1000 * sizeof(int)); if (buffer == NULL) { //  
Handle lack of memory gracefully return; }
```

8. Defensive Programming Techniques

Defensive programming is the practice of anticipating failure. It includes:

- **Asserting Assumptions:** Using `assert.h` to check conditions during development.
- **Input Validation:** Never trust user input. Check lengths of strings and ranges of numbers.
- **Read-Only Protection:** Using `const` for variables that shouldn't change.

9. Common Pitfalls

1. **Ignoring Return Values:** Functions like `scanf` or `printf` return values that indicate how many items were processed. Ignoring these is a common source of bugs.
2. **Checking for the wrong value:** Some functions return 0 for success, others return 0 for failure (like pointers). Always read the documentation (man pages).

10. Summary and Best Practices

- Check **every** system call return value.
- Use `stderr` for error messages instead of `stdout`.
- Free resources (files/memory) even when an error occurs.
- Use `errno` immediately after a failure, as subsequent calls may overwrite it.

Practice Exercise: Write a program that tries to open a file provided by user input and handles "Permission Denied" vs "File Not Found" differently.

Congratulations!

You have completed the Error Handling guide.

Stay connected with **CodeHive** for more C programming depth.

Ref ID: CH-ERR-2026-X